Freight Rate Prediction

Machine Learning Engineer assessment — modelling approach, validation design and results. 48,000 labelled loads (Jan–Oct 2025) are used to predict 12,000 loads in Nov–Dec 2025, plus a fixed-load December rate curve.

1. Summary

The model predicts the Sep–Oct forward holdout with a **median absolute error of 1.06%** ($19.80 on a typical load). On the 97% of holdout rows whose label passes a corruption screen it reaches **MAE $30.13, RMSE $44.54, MAPE 1.27%, R^2 0.9990**. Across all rows, including the ~1.3% whose labels are corrupted in the source data and are unpredictable by construction, MAE is $86.25 and R^2 is 0.830.

- **Target.** Log rate-per-mile. The rate is close to multiplicative in distance, equipment and the market, and the label corruption is multiplicative too.
- **Structure.** Two parts. A gradient-boosted model prices the load within its day; a four-coefficient regression places the day in the market. Only the second part has to extrapolate, and it is deliberately small enough to do so safely.
- **Split.** Forward in time, always. A random K-fold would put loads from the same day on both sides of the split and report a score the model cannot reproduce on a real future month.

2. The data

48,000 training loads spanning 2025-01-01 to 2025-10-31 and 12,000 validation loads spanning 2025-11-01 to 2025-12-31 — the validation set is strictly in the future, and it is two full months, not a random sample. Each load carries origin and destination (64 cities, each with one fixed lat/lon), distance, equipment (Dry Van 57%, Reefer 25%, Flatbed 18%), weight, date, and two market series: **market_index** and **quote_signal**.

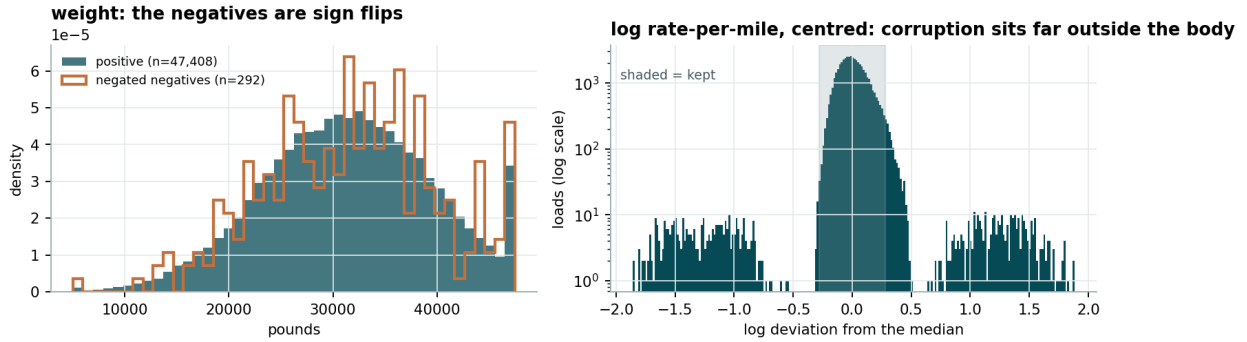
- **market_index is a daily market series.** Within-day sd is 0.025 against an overall sd of 0.166 — each day has a level, and each row a small deviation from it. Both matter: the day level carries a coefficient of 0.085 on log rate-per-mile, and a row one sd above its own day still runs ~0.3% dearer.
- **quote_signal is noisier but not noise.** Its daily mean moves the rate level with a coefficient of 0.047 (t = 13), and the row-level deviation adds a further monotone effect.
- **Distance is noisy, and the rate was generated from the noisy value.** The same lane shows ~2% sd in reported distance, but replacing distance with a de-noised per-lane median makes the fit *worse* (residual sd 0.0493 against 0.0456). So distance is used exactly as recorded.
- **12% of validation loads touch cities that never appear in training** (San Diego, Laredo, Jackson, Charlotte, Chicago, Knoxville, Norfolk, Allentown). Every geographic signal is therefore expressed in coordinates as well as in city names, so the model interpolates to them; the nearest known market is 1–230 miles away. Predicted rate-per-mile on those rows (mean 2.222) matches the rest of the set (2.211).

3. Data-quality issues, and what was done about them

Issue	Extent	Treatment
weight sign-flipped	292 train / 145 valid	absolute value — |w| matches the positive distribution exactly
weight missing	300 train / 165 valid	median impute + missing flag
market_index missing	374 train / 249 valid	same-day mean of the other loads, then global median
posted_rate corrupted	~1.3% of labels	detected out-of-fold, dropped before the final fit
August labels noisier	whole month	month down-weighted to 0.35 by inverse residual variance

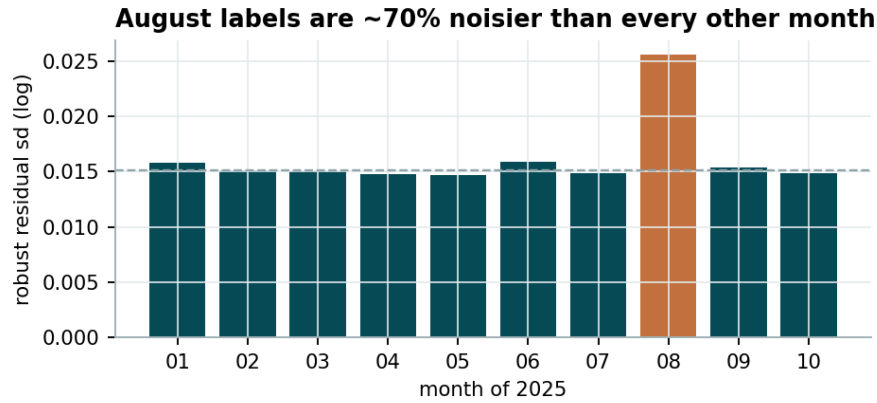
The last two are the ones that actually cost accuracy. **Label corruption:** about 1.3% of posted_rate values are multiplied by a random factor — roughly 1.4× to 6.6× up, or 0.15× to 0.7× down. That is one to two orders of magnitude larger than the genuine residual spread (sd ≈ 0.015 in logs), so the two populations separate cleanly.

They are flagged from the out-of-fold residuals of a date-blind model and dropped: 1.60% of training rows, leaving 47,234. Dropping is right rather than clipping, because the corruption carries no information about the true rate.



Left: negating the negative weights reproduces the positive distribution (n = 292, hence the jitter), which is what identifies them as sign flips rather than credits or returns. Right: log rate-per-mile has a tight body and two far-flung shoulders; the shaded band is what is kept.

August: residual spread in August is 0.026 against ~0.013 in every other month — uniformly across days, equipment types and haul lengths, so it is extra noise on the labels rather than anything a feature explains. Nothing repairs that, but it is heteroskedasticity, so each month is weighted by the inverse of its residual variance. August lands at 0.35, and held-out cross-sectional error improves 2–5%. It also means the August holdout below is a poor guide to model quality, and it is read as such.



Robust residual sd by month, from an out-of-fold cross-sectional fit. August stands alone.

4. Model

Variance in log rate-per-mile splits into a within-day part (sd ≈ 0.018 — which load is it?) and a day-level part (sd ≈ 0.030 — what was the market doing?). The two need different machinery, so they get it.

4.1 Cross-section — gradient boosting

LightGBM on log rate-per-mile, 63 leaves, learning rate 0.03, feature fraction 0.7, 1,500 rounds, averaged over four seeds. Features: distance and log distance, weight and weight-per-mile, equipment, both cities as categoricals, both coordinate pairs plus midpoint, deltas, great-circle distance, detour ratio and bearing, lane frequency, day-of-week and day-of-month, and the row's deviation from its own day's market index and quote signal. Everything is tuned against *within-day* error, which is free of the day-level forecasting problem and robust to the corrupted labels.

4.2 Day level — four coefficients, fitted by weighted least squares

This is the only part that extrapolates, and the design is driven by that. Trees cannot extrapolate a trend at all; worse, when handed day aggregates such as the daily mean quote signal they memorise *which month* a day belongs to and then fan out on a new month whose aggregates sit in a narrow band. That was measured, not assumed: giving the trees the daily quote mean tripled the median error on an August holdout — and November and December have exactly the narrow-band shape that triggers it.

So the day level is a linear regression on the daily mean of the cross-sectional model's out-of-fold residual, weighted by each day's load count and by its month's reliability:

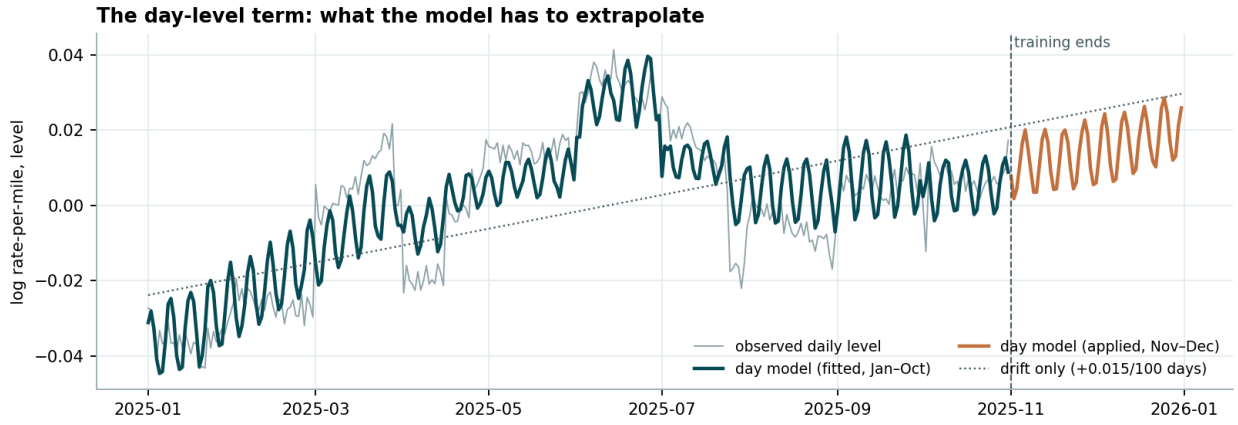
Day-level term	Coefficient	robust t
log(market_index), daily	+0.0845	29.3
quote_signal, daily mean	+0.0472	12.5
calendar time (per day)	+0.00015	24.3

The calendar-time coefficient is the secular drift: **+0.0147 log-units per 100 days**, about +5.5% a year, on top of everything the market index explains. It is what makes the Nov–Dec level right. Zero that one coefficient and the same model, refitted, predicts the average of its training window instead: on the Sep–Oct holdout the median bias goes from –0.51% to –2.40% and MedAPE more than doubles, from 1.06% to 2.44%. A window two months past the training data, like the real one, is further out still. Out-of-fold residuals are used to fit it because in-sample GBM residuals are shrunk and would bias the drift toward zero.

The specification was chosen by backtesting the daily level series on its own (`experiments/day_level_backtest.py`): roll an origin forward every week, fit on everything before it, forecast the next 30 and 61 days, score. Three things come out of it, all quoted below at 61 days — the real horizon — and averaged over the origins with seven months or more of history, which are the ones that resemble the real task.

- **Calendar time is doing the work.** Level RMSE 0.0102 with it, 0.027–0.029 without — a factor of three, and nothing else recovers it.
- **Day-of-week and day-of-month belong in the trees, not here.** Adding them to the day model wins on the average over all twenty origins (0.0124 against 0.0197), but that average is carried by the June–July origins where every specification is poor. At the realistic origins they are the worst of the set (0.0130 against 0.0102), and the gap opens up with horizon — at 30 days they are level with it. Their coefficients are not stable enough to carry two months.
- **The quote term is a genuine coin flip.** Dropping it scores 0.0098 against 0.0102 at the realistic origins, and 0.0118 against 0.0116 on the median over all of them. It is kept because it is strongly significant in-sample and costs nothing either way: November and December sit almost exactly at the training mean of the quote series, so it moves the final level by about 0.02%.

The error also does not grow with horizon — 0.0105 at 30 days against 0.0102 at 61 — which is the evidence that the drift is a genuine linear trend rather than a local slope that happens to fit the end of the training window.



The grey line is the observed daily level, the dark line the fitted day model, the orange line the same four coefficients applied to Nov-Dec using the market and quote series supplied for those dates. The dotted line is the drift alone.

5. Validation and data split

Every split is forward in time. Once distance, equipment and geography are accounted for, most of what remains is a day-level market effect (sd 0.030, against 0.018 within a day), so a random K-fold would put loads

from the same day on both sides of the split and hand the model much of the answer; it would also hide the drift, since each fold would contain the full date range. Neither failure would show up in the K-fold score, and both would show up in production.

- **Primary — forward holdout.** Train Jan–Aug, score Sep–Oct. Two unseen months, the same horizon as the real Nov–Dec task.
- **Stability — rolling origin.** Four successive 1-month holdouts (Jul, Aug, Sep, Oct), each trained only on what precedes it.
- **Inside the training fold** the model fits its own out-of-fold residuals twice — once to flag corrupted labels, once to build the day-level series. Both use random folds, which is safe precisely because neither model carries a day-level feature, so rows from the same day cannot leak a level to each other.
- **Model selection was never done on the final validation set**, which has no labels. Cross-sectional choices were scored on within-day error; the day-level specification was scored on its own rolling forecast backtest.

One note on what is shared across the split. Daily market aggregates, the city coordinate table, lane counts and the category lists are computed over the training and validation *feature* columns together. No label is involved, and the validation features are given as part of the task; this is what makes the December chart possible at all, since those 31 rows arrive with no market data of their own.

6. Results

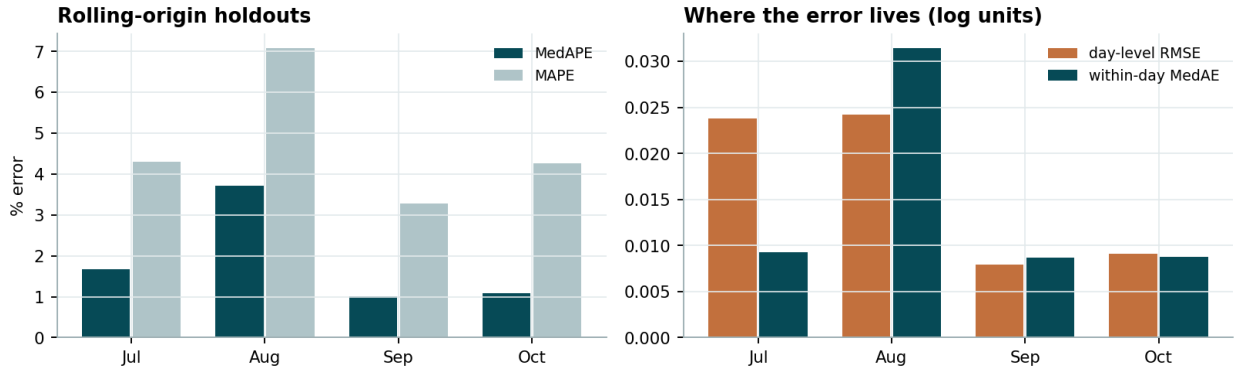
Sep–Oct forward holdout, 9,523 loads. The two rows differ only in whether the ~1.3% of rows with corrupted labels are included. The screen is built inside the holdout from its own loads and labels, never from the model being scored — and MedAPE barely moves (1.064% to 1.046%), which is how you can tell it is removing bad labels rather than flattering the model. The mean-based metrics move a great deal, because a handful of 6× labels dominate a squared error.

Sep-Oct holdout	MAE	RMSE	MAPE	MedAPE	R^2
all 9,523 rows	$86.25	$628.91	3.79%	1.06%	0.830
labels passing the screen (9,248)	**$30.13**	**$44.54**	**1.27%**	**1.05%**	**0.9990**

Rolling-origin backtest, median absolute percentage error on every row, and the same error split into its day-level and within-day halves:

Holdout month	MedAPE	MedAE	median bias	day-level RMSE	within-day MedAE
July	1.68%	$32.16	+0.26%	0.0238	0.0093
August	3.72%	$66.68	+2.22%	0.0242	0.0315
September	1.01%	$18.68	-0.30%	0.0079	0.0087
October	1.09%	$20.56	-0.57%	0.0091	0.0088

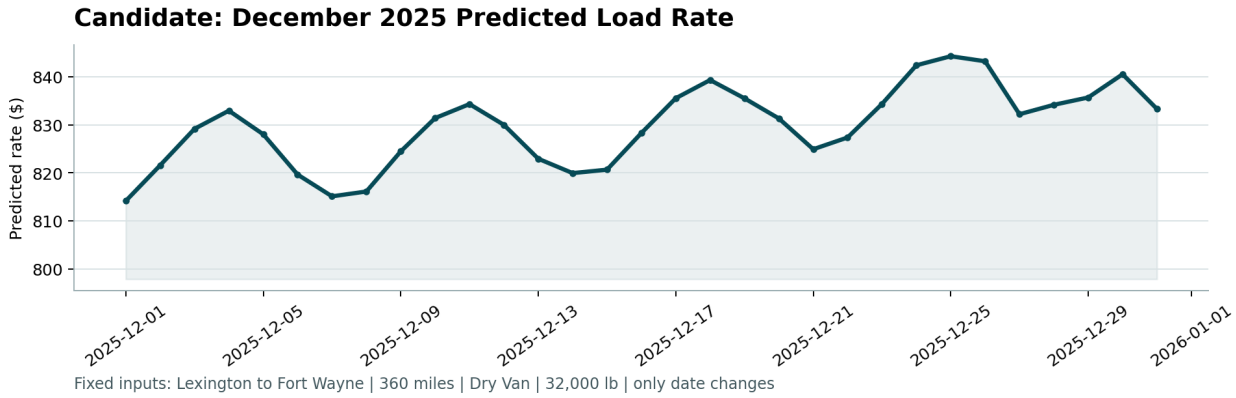
September and October — the two origins with the most history, and so the closest analogues to the real task — land at 1.01% and 1.09% with biases under 0.6%. July's day level is harder: fitted on only six months, the drift estimate is distorted by a mid-year bump that later reverts. August is the noisy month described in section 3; its within-day error (0.031) is three times every other month's and is a property of the labels, not the model. Both effects shrink as history grows, and the final model has ten months of it.



Left: error by holdout month. Right: the same error split in two. The within-day half is flat and small except in August; the day-level half is what improves as the training window lengthens.

7. The fixed December chart

Thirty-one identical loads — Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb — one per December day. Only the date changes, so the curve *is* the day-level term, and it is a direct read-out of the model's view of the December market. Coordinates come from the city table; market_index and quote_signal come from the daily means of the validation loads on those dates, which is where the task supplies December market conditions. The lane itself is well attested: 44 loads in the data, reported distances averaging 365 miles, so 360 needs no extrapolation.



Produced by the provided score.py. Predicted rates run $814.16 to $844.30.

The shape is a weekly cycle on a slight upward drift. Mid-week peaks and weekend troughs come from market_index, which carries a strong weekly pattern; the month-over-month rise is the drift term. Both are properties the model learned from ten months of data, not assumptions imposed on December.

8. Limits, and what I would do next

- **The drift is one year of one series.** Ten months cannot separate a secular trend from slow annual seasonality; the linear term is the defensible reading and the backtest supports it out to 61 days, but a second year of history would settle it. The genuinely extrapolated part is small — past the last training date the drift adds 0.46% by 1 December and 0.90% by 31 December — so even a badly wrong slope costs under a percent. The larger job the term does, placing the level correctly *within* the fitted range, is interpolation and is well identified.
- **The day level still has 0.008–0.010 of unforecastable wobble.** That is now comparable to the entire within-day error, so it is where the next real gain is — most likely from market features the dataset does not contain (fuel, capacity, holidays) rather than from more modelling of what is here.
- **Predictions are conditional medians.** With residual sd ~0.015 the median-to-mean gap is ~0.01%, so it is not worth a smearing correction. If the scorer used a squared error over the raw, still-corrupted labels, a ~1.4% uplift would score marginally better; that is fitting the corruption rather than the freight market, so it was left out deliberately.

- **Unseen cities are interpolated, not learned.** Coordinates carry them, and their predicted rate-per-mile matches the rest of the set, but a genuinely new region far from any training city would be an extrapolation the validation here cannot vouch for.